ASSEMBLY LANGUAGE
BCT 2302
MALWARE ANALYSIS

- High Level Assembler (HLA) → Makes it easier to learn assembly language.
- ✓ HLA combines the syntax of a high level language with the low level programming probabilities of assembly language

Obtaining Materials for the Course

- All the software is available from the Webster web site.
- URL is http://webster-cs.ucr.edu
- You can also use a google search using "Art of Assembly" to locate the current home site of this material.

Anatomy of an HLA Program

HLA program takes the following form

① program pgmID ①
Declarations ②
④ begin pgmID;
statements ③
④ end pgmID;

1) Specifies name of the program

2) Is the declaration section where you declare constants, types, procedures and other objects in the program.

3) This is where you place the executable statements for your main program.

④ This are reserved words that delinate the program

N/B: Program ID is a user defined program identifier
HLA identifier are Case neutral / Case sensitive

Hello World program

```
program helloWorld; ④
① #include ("stdlib.hhf"); ④
  begin helloWorld;
② stdout.put ("Hello, MMU Students", nl); ③ ④
  end helloWorld; ④
```

HLA -V ugali
ugali

Components of the Hello World Program

1) #include tells the HLA compiler to include a set of ~~compilat~~ declarations from the standard library header file (stdlib.hhf)
2) stdout.put it is the "print" statement for the HLA language.
3) The semicolon follows the program Begin stdout and End statement, however it is not necessary after the #include statement

Some basic HLA Data Declarations

- HLA provides constants, type and data declarations statements.
- HLA provides 3 different integer types

int 8 —Corresponds→ 8 bits (one byte)
int 16 ——→ 16 bits (two byte)
int 32 ——→ 32 bits (four byte)

- Declarations happens in the HLA static section. ④

Example → Keyword → data type

```
static
    i8 : int8;
    i16 : int16;
    i32 : int32;
```

↳ Names of variables

In the ~~Starting~~ static section You can also give a variable an initial value.
The OS will assign the ~~value~~ variable with the value when it loads the program.

Boolean values

- To declare boolean variables use the boolean datatype
- HLA uses a single byte to represent boolean values.

Example

```
static
    BoolVar : boolean;
    HasClass : boolean := false;
    IsClear : boolean := true;
```

Note: You may declear initialized and ~~uninit~~ ~~or~~ ~~uninit~~ uninitialized variables.

Character Values

- HLA allows you to declare one-byte ASCII characters with the char datatype
- You may initialize character variables

example

```
static
    C: char;
```

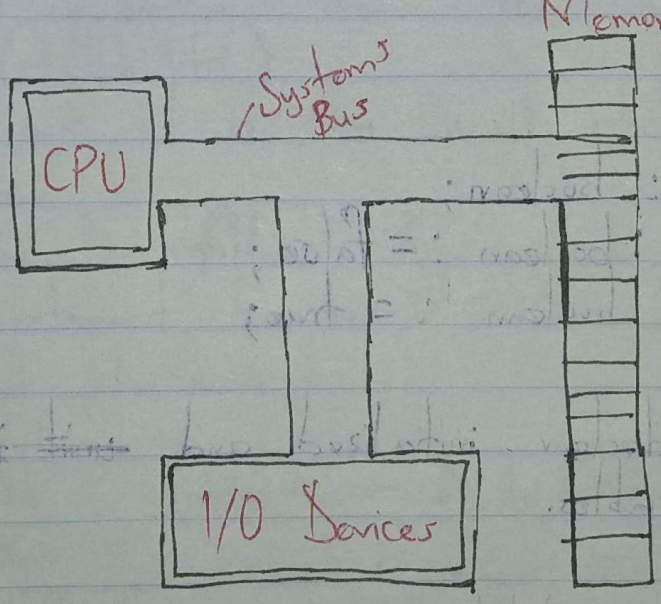
variable	data type	const
i32	int 32	-32

Example

```
Static
    C: char;                    → Pair of apostrophes
letterA: char: 'A';
```

Introduction to the Intel 80x86 CPU Family

- You need to understand the basic structure of the intel 80x86 family to appreciate machine instructions
- The intel CPU family is generally classified as Von Neuman Architecture Machine.
- This computer system consist of 3 main building blocks;
  i) CPU
  ii) Memory
  iii) I/O devices
- This components are connected together using a system bus



✓ The most prominent items within the CPU are registers
✓ The intel CPU registers can be broken down into four categories;
i) General purpose registers
ii) Special purpose application accessible registers
iii) Segment registers
iv) Special purpose kernel mode registers.

Note: In this course we will consider the first two sets of registers

i) 80x86 General purpose Registers

This include;

- eight 32-bit registers that have the following names

(32)
EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP

- the E prefix stands for extended
- This prefix differentiates the 32-bits registers from the other registors

- 16-bit registers have the following names

(16)
AX, BX, CX, DX, SI, DI, BP, SP

- The 80x86 CPU provides eight 8-bit registors that have the following names;

(8)
AL, AH, BL, BH, CL, CH, DL, DH

Some Basic Machine Instructions

MOV instructions

- Most often used assembly language statement.
- HLA Syntax

mov(source_operand, destination_operand);

source_operand: Register, Variable, Constant

destination_operand: registor, Variable

- Both operands cannot be memory variables.
- Both operands must be of the same size.

Legal 80x86 MOV Instructions Operands

Source	Destination
Reg_8	Reg_8
Reg_8	Mem_8
Mem_8	Reg_8
Constant	Reg_8
Constant	Mem_8
Reg_{16}	Reg_{16}
Reg_{16}	Mem_{16}

The 80x86 ADD and SUB instructions

- Allow add and subtract operands.
  Syntax is identical to the MOV instructions.

```
add (source_operand, destination_operand);
sub (source_operand, destination_operand);
```

- They take the same form as the move instruction
- Add instruction does the following;
  destination_operand = destination_operand + source operand;
- Sub does the following;
  destination_operand = destination_operand − source operand

Basic HLA Control Structures

- You require the ability to make decisions and create loops in HLA programs.
- Use high level control structures
- These include:
  - IF statement
  - While statement
  - For statement
  - Repeat
  - Forever

Boolean Expressions in HLA Statements

Several HLA statements require a Boolean expression to control their execution.

Examples include:

i) IF statement
ii) While statement
iii) Repeat statement

A Boolean expression takes the following form

register
!register
Boolean_variable
!Boolean_variable

Examples of legal Boolean expressions

destination operand / source operand

EAX < EBX
EBX > 5
i32 < -2
i8 > 128
Bool_Var
al
ESI

Note:- A register operand can be any of the 16 bit, 32 bit or 8 bit general purpose registers.
- In the case where the operand is only one register, the expression evaluate false. If the register contains a zero and it evaluates true if the register evaluates a non zero.
- If you specify a boolean variable as the expression, the program tests it for zero (false) or Non Zero (True)
- HLA uses the value zero & one to represent FALSE and TRUE

- Both operands cannot be memory variables
- Left operand is the source operand and the Right operand is the destination operand.
- Both operands must be of the same size.
- If the right operand is a constant, its value must be in the range that is compatible with the left operand.

HLA IF statement

It uses the following syntax

```
if(expression) then
    statements
elseif clause optional
elseif(expression) then
    statement(s)
else clause optional
else
    statements
endif;
```

- Use of a Single IF... THEN clause.

Since ELSEIF and ELSE clauses are optional, and IF statement could take the form of a single IF... THEN clause followed by a sequence of statement and a closing END IF clause.

Example

```
if (eax = 0) then
   stdout.put("there is an error", nl);
endif;
```

IF statement with a single else clause and if statement with an optional clause.

```
if (eax = 0) then
   stdout.put("there is an error", nl);
else
   stdout.put("there pointer is correct", nl);
end if;
```

A sequence of ESEIF clauses

- You can create sophisticated decision making logic by including the Else If clause into an IF statement

```
if (ch = 'a') then
   stdout.put("You selected the 'a' menu", nl);
else if (ch = 'b') then
   stdout.put("You selected the 'b' menu", nl);
else if (ch = 'c') then
   stdout.put("You selected the 'c' menu", nl);
```

```
else
stdout.put("Error : illegal menu selection" nl);
endif;
```

The WHILE... ENDWHILE Statement

- Basic syntax (boolean expression)

```
while (expression) do
    statements
endwhile;
```

- It evaluates a boolean expression
If it is false, control immediately transfers to the statement after the loop.
- If true, control moves through the body of the loop.

example

```
mov(0,i);
while(i<10) do
  stdout.put("i=", i, nl);
  add(1,i)
endwhile
```

The FOR... ENDFOR Statement

It takes the following general form on the syntax

```
for (initial_stmt; Termination Expression; Post_Body_Statement) do
    Statement
endfor;
```

Initial statement - It initializes the register or a memory variable with a zero or some other value

Termination Expression - It is a boolean exp. determines whether the loop will execute
Part-body-statement - It modifies the value of the loop, control variable

Example:

```
for (mov (0,i); (i<10); add (1, i)) do
    stdout.put ("i=", i, nl);
endfor;
```

The REPEAT...UNTIL Statement

- Uses the following syntax

```
repeat
statements
until (expression);
```

Example

```
mov (10, eax)
repeat
stdout.put ("eax=", eax, nl);
sub (1, eax);
until (eax=0);
```

The BREAK and BREAKIF Statements

- They provide the ability to prematurely exit from a loop
- Syntax

```
break;
breakif (expression);   ,boolean
```

The FOREVER... ENDFOR Statement

- Uses the following syntax

```
forever
    statements;
endfor;
```

- It creates an infinite loop
- It may use the break and breakif statements to create a loop that tests determination of the loop in the middle.

Example

```
forever
    stdout.put( "Enter an integer less than 10: ");
    stdin.get(i);
    breakif( i > 10);
    stdout.put( "The value needs to be less than 10!", nl);
endfor;
```

The HLA Standard Library

There are two reasons HLA is much easier to learn and use than standard assembly language;

i) HLA's high level syntax for declarations and control structures

ii) The HLA standard library provides commonly needed, easy to use, assembly language routines that you can call without having to write this code yourself.

- The HLA standard library consist of several modules organised by category. See table 3

Predefined Constants in the STDIO Module

Let us describe some common constants that the stdio module defines for you.

- nl - Stands for newline
- stdio.bell - The ASCII bell character. Beeps the speaker when printed
- stdio.bs - The ASCII backspace character
- stdio.tab - The ASCII tab character
- stdio.newln - Prints out a newline sequence in the stdout device.
  - It is equivalent to saying ⇒ stdout.put(nl);

STDout.putiX Routines

- They print a single parameter as an integer value.
- These are the following;

```
stdout.puti8
stdout.puti16
stdout.puti32
```

- The parameter maybe a constant, a register or a memory variable.
- The actual parameter must be the same size as the formal parameter

Example

```
stdout.puti8(123);
stdout.puti16(DX);
stdout.puti32(mm4);
```

Stdout.putiX Size Routines

They output signed integer values to the standard output. This routine let you specify the number of print positions the value will require on output

They require the following parameters;

```
stdout.put i8 size (value8, width, padchar);
stdout.put i16 size (value16, width, padchar);
stdout.put i32 size (value32, width, padchar);
```

↑ constant, Register, Variable
↑ Integer constant -256 & +256, Register, Memory location

- The width parameter can be an integer constant between -256 and +256. This can be a constant register or a memory location.
- The padchar should be a single character value.

Example

```
stdout.put i16 size (AX, 5, *);
```

The stdout.put Routine

- The generic form is as follows

```
stdout.put (value_to_output);
```

- The parameters consist of a list of one or more constants, registers or memory variables separated by commas.
- Each parameter can take (X one) of the following forms.

```
stdout.put (value);
stdout.put (value, width);
stdout.put (value1, value2);
stdout.put (value1 : width, value2 : width);
```

Stdin.getc Routine

This routine reads a character from the std input device and stores it in the al register

Data Organisation

80x86 chip

Bit

- It is the smallest unit on a binary computer.
- It takes two different values, zero or one.
- You can use a bit to represent two distinct items, e.g zero or one, true or false, on or off, male or female, right or wrong

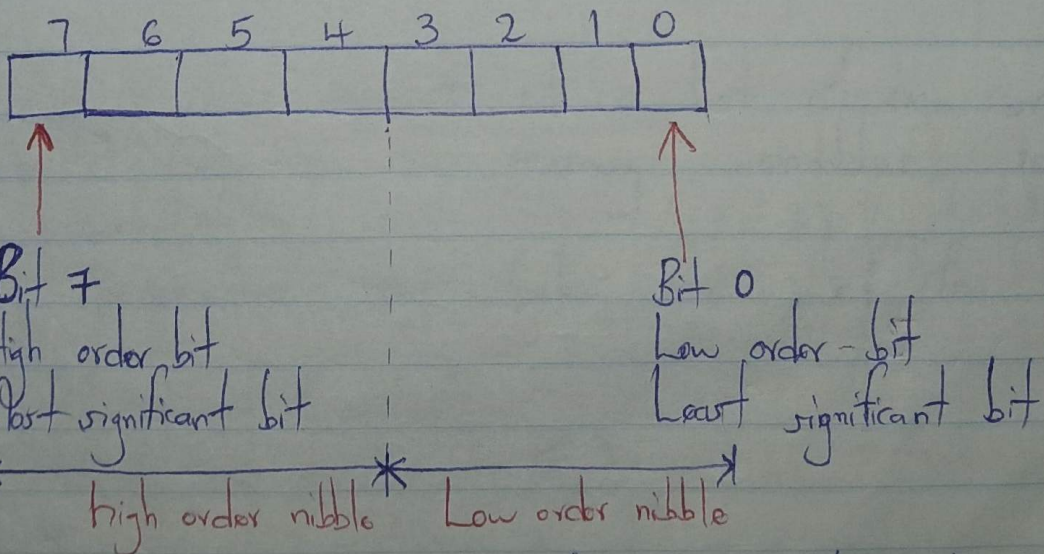
Nibble

It is a collection of four bits.

example
0000
0001
0010
0100
1000

Byte

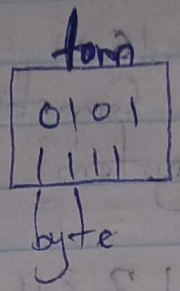
- Consist of 8 bits
- It is the smallest addressable data item(datum) in the 80X86 microprocessor



- If you want, you can create a variable whose data type is byte.

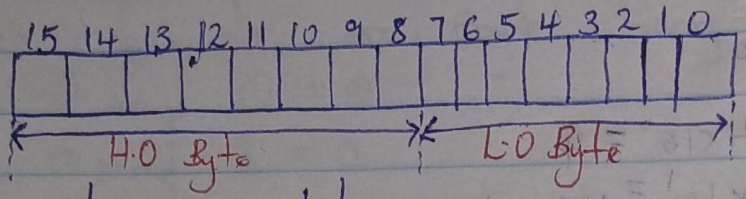
Example

```
Static
  tom: byte;
```



Words

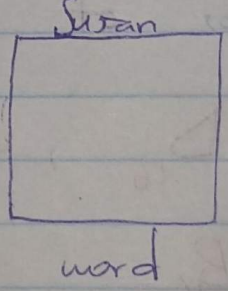
A word is a group of 16 bits



- It contains 2 bytes
- If you want, you can create a variable whose data type is one.

Example

```
Static
  Susan: word;
```



Double words

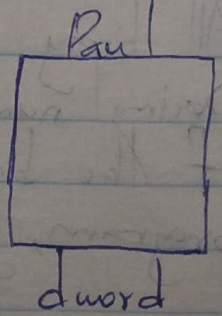
Contains a pair of words

It is 32 bits long.

If you want, you can create a variable whose data type is dword

Example 2

```
Static
  Paul: dword;
```



The Hexadecimal Number System

The Base is 16.

Note: Most Computer systems engineers use hexadecimal number system

Example

1234_{16}

Converting to decimal:

$1^3 2^2 3^1 4^0{}_{16}$

$4 \times 16^0 = 4$

$3 \times 16^1 = 48$ +

$2 \times 16^2 = 512$

$1 \times 16^3 = 4096$

4660_{10}

- Each hexadecimal can represent values between 0 – 15
- Examples of values in hexadecimal numbers;

$FEED_{16}$, $BEEF_{16}$, $DEAD_{16}$, $DEAF_{16}$, 1234_{16},

$0AFB_6$

Note; We sometimes need to enter hexadecimal numbers into the computer systems, therefore we need a different mechanisms for representing hexadecimal numbers, we therefore adopt the following convention;

i) All hexadecimal numbers begin with $, example $1234
ii) All binary numbers begin with % sign, example %0101
iii) Decimal numbers, do not require a prefix character
iv) If the base/radix is clear on the context of the program, you may drop the leading $/% character

Logical Operations On Bits

- There are four main Logical operations which you can perform on hexadecimal & Binary numbers.
- These are;

AND, OR, XOR, NOT

AND

- It is a dyadic operation i.e it accepts exactly two operands.
- This operands are single binary bits.

0 and 0 = 0
0 and 1 = 0
1 and 0 = 0
1 and 1 = 1

OR

- It is also a dyadic operation.
- Defined as follows;

0 or 0 = 0
0 or 1 = 1
1 or 0 = 1
1 or 1 = 1

XOR

- It is also a dyadic operation
- Defined as follows

0 XOR 0 = 0
0 XOR 1 = 1
1 XOR 0 = 1
1 XOR 1 = 0

NOT

- It is monadic operation
- It accepts only one operand.

NOT (0) = 1
NOT (1) = 0

Example

Perform a logic AND operation on the following numbers

```
  %10110111
AND %11011110
  %10010110
```

- The 80x86 CPU supports this bitwise logical operations using four instructions that apply to operands.
- This instructions are;
  AND, OR, NOT, XOR
- Those instructions, use the same syntax as the ADD and SUB instructions

```
and (source, dest);
or (source, dest);
XOR (source, dest);
```

source: Constant / Variable / Register
dest: Variable / Register

- Both operands must be of the same size.
- The 80x86 Logical NOT has a single operand
  Syntax:

```
not (dest);
```

- The destination operand must be a register or

memory variable operand.

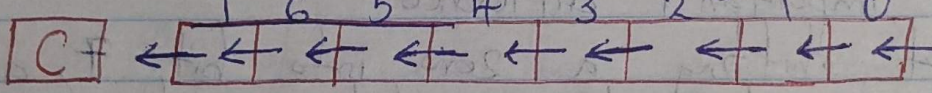
Note: This instruction inverts all the bits in the specified destination operand.

Shifts and Rotates

- This is another set of logic operations which apply to strings of bits.
- Broken down into;
  - left shifts
  - left rotate
  - right shift
  - right rotate

Left shift

- It moves each bit in a bit string, one position to the left



- Bit zero moves into position 1 and bit 1 moves into position two, and so on..
- The previous value of bit 7 is the carry out operation
- We shift a zero into position zero.

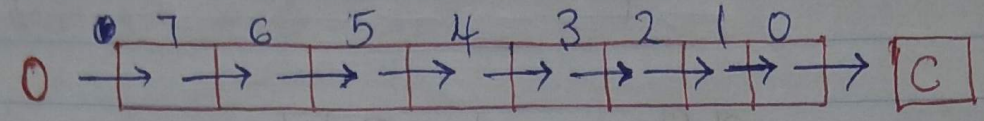
Syntax

ShL (count, dest);

- The count operand is a constant in the range 0--- n, where n is 1 less than the number of bits in the destination operands. i.e n = 7 for 8 bit operands n = 15 for 16 bit operands and n = 31 for 32 bit operands
- The destination operand can either be a memory variable or a register.
- C represents the Carry flag

Right Shift

- Shift/moves each bit in a bit string one position to the right



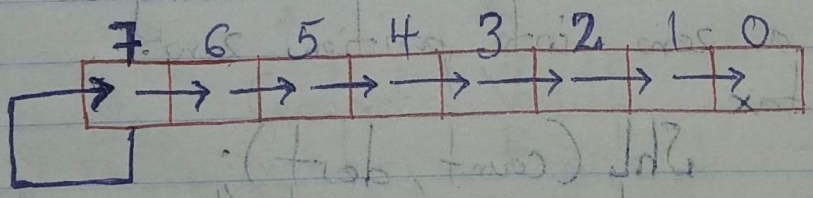
- Bit 7 moves to bit 6, and so on
- We move a 0 into the bit 7.
- Bit 0 goes into the carry

Syntax:

```
SHR (Count, dest);
```

Arithmetic Shift Right

- It works just like the normal shift right operation with one important difference.
- Instead of shifting a zero into bit 7, arithmetic shift right operation leaves bit 7 alone.
- i.e, during the shift operation, it does not modify the value of bit 7.



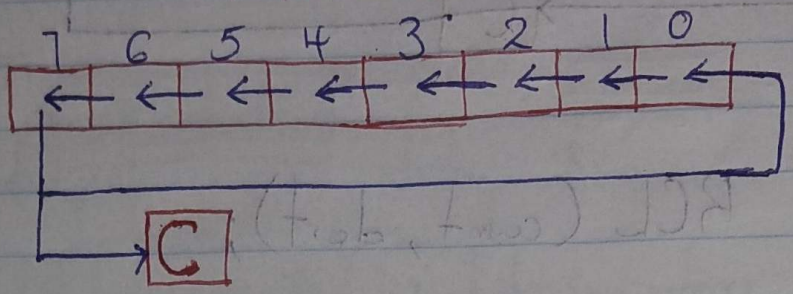
Syntax

```
SAR (count, dest);
```

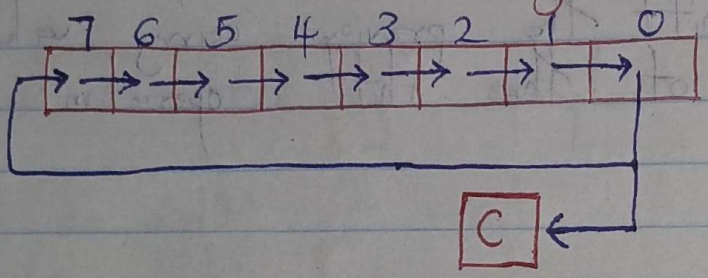
Rotate Left & Rotate Right

- They behave like the shift left and shift Right operations with one major difference
- The bit shift from one end, is shifted back in at the other end.

Rotate Left



Rotate Right



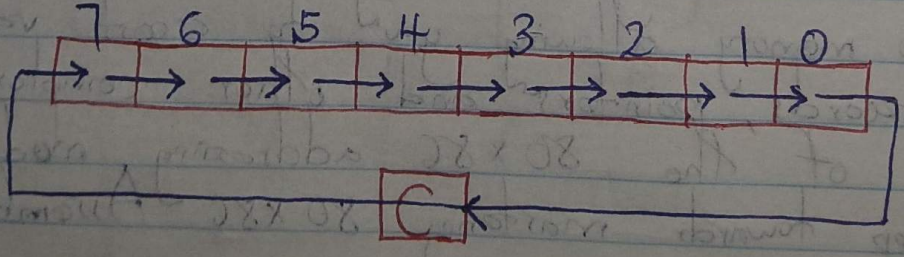
Note: The two instructions also copy the bit from the destination operand to the carry flag.

Syntax

```
rol (count, dest);
ror (count, dest);
```

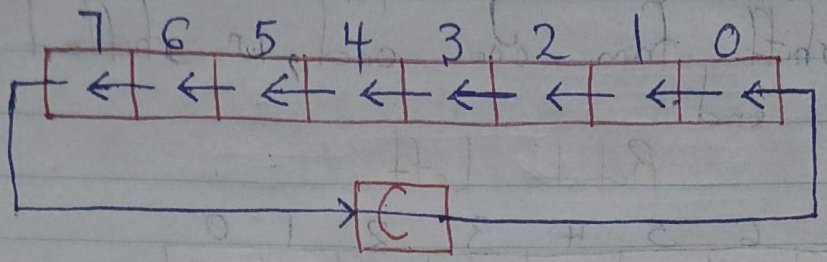
Rotate Through Carry Right And Rotate Through carry left

Rotate Through Carry Right.



Summary of Vol 1 chapter 4

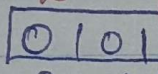
Syntax RCR (count, dest);



Syntax RCL (count, dest);

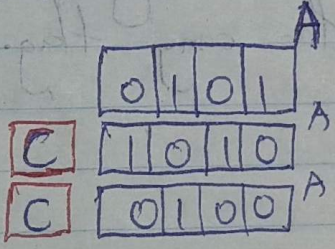
- This operations shift the out put bit through the carry and shift the previous carry value back into the input of the shift operation.

Example



ShL (2, A) = 0100

ShR (2, A) = 0001



Memory Access and Organization

- The 80x86 processor, lets you access memory in many different ways.
- The 80x86 memory addressing modes, provides flexible access to memory allowing you to access variables, arrays, records, pointers and other complex datatypes
- Mastery of the 80x86 addressing modes is the first step towards mastering 80x86 Assembly Language.

80x86 Register Addressing Modes

- Most 80x86 instructions operate on the 80x86 general purpose Register set.
- By specifying the name of the register, as an operand to the instruction, you may access the contents of the Register.

Example:

mov (source, destination);

- This instruction copies data from the source operand to the destination operand.
- The 8 bit, 16 bit & 32 bit registers are valid operands for this instructions.

- Consider the following examples;

```
mov (bx, ax);
mov (al, dl);
mov (edx, esi);
mov (bp, sp);
mov (cl, dh);
mov (ax, ax);
```

80x86 Memory Addressing Modes

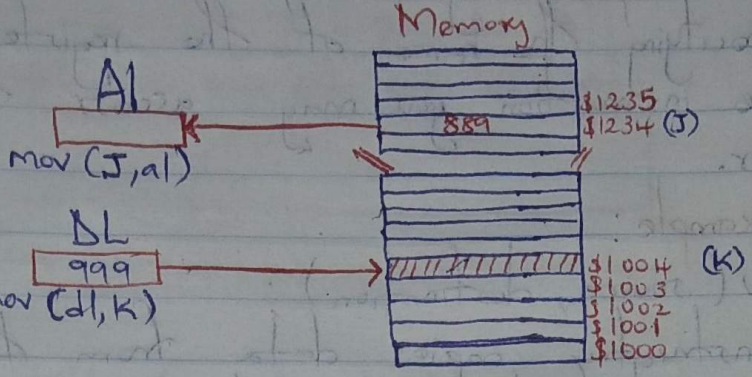
- The 80x86 provides many different

The addressing modes provided by the 80x86 family of CPU's include:

i) The displacement only addressing mode
ii) The register indirect addressing mode.
iv) Index addressing modes
iii) Scale indexed addressing modes.

The displacement Only Addressing Mode

- Most commonly used addressing mode.
- It is the easiest to understand.



- The displacement only addressing mode is perfect for addressing simple scalar modes.

For example;

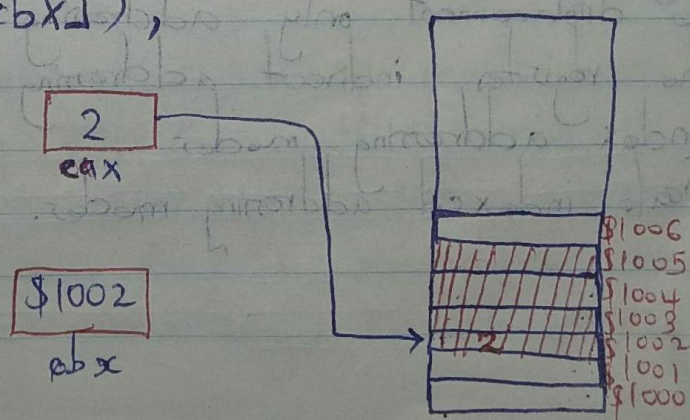
- In the first item, loads AL register, with a copy of the byte at memory location $1234
- In the second item, stores the value of the DL register to memory location $1004

The Register Indirect Addressing Modes

- The 80x86 CPU allows you to indirectly access memory through a Register.
- The term indirect means the operand is the actual address but rather the operands value specify the memory address to use.

Example

```
mov(eax, [ebx]);
```



- In the case of register indirect addressing mode, the register is the memory location
- Tells the CPU to store eax value at the location whose address is in ebx.
- The register indirect addressing mode has lot of uses.
  i) You can use it to access data referenced by a pointer.
  ii) You can use it to step through array data.
  iii) You can use it whenever you need to modify the address of a variable while your program is running.

Note: Address operator & (address of)

Example:

```
mov(&J, ebx);
```

Indexed Addressing Modes

The Stack Segment and the Push and Pop Instructions

- The stack segment in memory is where the 80x86 maintains the stack.
- The stack is a dynamic data structure that grows and shrinks according to certain memory needs of the program.
- The 80x86 controls it's stack via the ESP register (Pointer).
- When you program starts execution the O.S initiates ESP with the address of the last memory location in the stack memory segment.
- Data is written to the stack segment by pushing and popping
- When you push data into the stack, the 80x86 decrements the pointer by the size of the data you are pushing and then it copies the data to memory where ESP is then pointing.

The Basic PUSH Instruction